

UNIVERSITAS NEGERI YOGYAKARTA
FAKULTAS TEKNIK
PROGRAM STUDI TEKNIK INDUSTRI

LAPORAN PRAKTIKUM
APLIKASI WEB DAN MOBILE

Semester Genap 2025/2026

**MODUL 10 – State, Hooks (useState, useEffect), dan
Conditional Rendering**

Nama Mahasiswa : Saprina Melita Sari
NIM : 23051430012
Kelas : A1
Tanggal Praktikum : 28 April 2026
Dosen Pengampu : Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT.

Dikumpulkan paling lambat 7 hari setelah pelaksanaan praktikum

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 10
Topik Utama	State, Hooks (useState, useEffect), dan Conditional Rendering
Sub-Topik	State Management, Lifecycle Hooks, dan Rendering Bersyarat.
Durasi Praktikum	340 Menit
Tanggal Praktikum	28 April 2026
Nama Mahasiswa	Saprina Melita Sari
NIM	23051430031
Kelas	A1

B. TUJUAN PEMBELAJARAN

1	Memahami apa itu State dan perbedaannya dengan Props.
2	Menguasai penggunaan `useState` untuk menyimpan data yang berubah-ubah.
3	Memahami `useEffect` untuk menangani side effects (seperti timer atau API call).
4	Mampu melakukan Conditional Rendering (menyembunyikan / menampilkan elemen berdasarkan kondisi).

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging

3	Node.js & npm	Node 18+ / npm 9+	Runtime JS (Modul tertentu)
4	Prettier	Versi 12.3.0	Extensions VS Code untuk merapikan indentansi code

D. DASAR TEORI

D.1 Konsep Utama

Konsep utama dalam materi ini mencakup state, hooks, dan conditional rendering pada aplikasi berbasis JavaScript modern. State berfungsi sebagai tempat penyimpanan data yang dapat berubah selama aplikasi berjalan, seperti jumlah produksi, status mesin, atau durasi proses. Ketika nilai dalam state berubah, tampilan aplikasi akan otomatis diperbarui tanpa perlu melakukan refresh halaman, sehingga aplikasi menjadi lebih interaktif dan responsif.

Hooks seperti `useState` dan `useEffect` digunakan untuk membantu pengembang dalam mengelola perilaku komponen. `useState` digunakan untuk mendeklarasikan dan memperbarui data, sedangkan `useEffect` berfungsi untuk menjalankan proses tambahan seperti pengambilan data dari server atau perhitungan ulang ketika terjadi perubahan. Dengan adanya hooks, struktur logika program menjadi lebih rapi dan mudah dikelola.

Conditional rendering merupakan teknik untuk menampilkan atau menyembunyikan elemen berdasarkan kondisi tertentu. Misalnya, ketika mesin dalam keadaan aktif maka sistem menampilkan status “Aktif”, sedangkan ketika mesin berhenti akan ditampilkan status “Maintenance”. Teknik ini membuat tampilan aplikasi lebih sesuai dengan kondisi nyata di lapangan.

D.2 Keterkaitan dengan Teknik Industri

Dalam konteks Teknik Industri, state dapat dianalogikan sebagai data operasional yang selalu berubah, seperti output produksi, waktu operasional mesin, atau jumlah produk cacat. Data tersebut perlu terus diperbarui agar pengambilan keputusan oleh manajemen dapat dilakukan secara tepat.

Hooks memiliki hubungan dengan proses pemantauan dan pengendalian sistem. Sebagai contoh, perubahan kondisi mesin dalam sistem produksi harus segera terdeteksi dan ditindaklanjuti. `useEffect` dapat diibaratkan sebagai mekanisme otomatis yang aktif ketika terjadi perubahan, seperti sistem peringatan saat terjadi gangguan atau penurunan performa.

Sementara itu, conditional rendering berkaitan dengan proses pengambilan keputusan berbasis kondisi dalam Teknik Industri. Contohnya, sistem hanya menampilkan notifikasi ketika nilai efisiensi berada di bawah batas standar. Dengan pendekatan ini, informasi yang disajikan menjadi lebih relevan dan mudah dipahami oleh pengguna.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Membuat Komponen Counter Produksi

Deskripsi singkat apa yang dilakukan pada langkah ini:

```
/* ===== KODE / OUTPUT LANGKAH 1 ===== */
import React, { useState } from 'react';
function CounterProduksi() {
  // Deklarasi State: 'jumlah' bernilai awal 0, 'setJumlah'
  fungsi untuk mengubahnya
  const [jumlah, setJumlah] = useState(0);
  const [target, setTarget] = useState(100);
  // Fungsi menambah produksi
  const tambahProduksi = () => {
    setJumlah(jumlah + 1);
  };
  // Fungsi reset
  const reset = () => {
    setJumlah(0);
  };
  return (
    <div className="text-center p-4 border rounded bg-light">
      <h3>Simulasi Hitung Produk</h3>
      <h1 className="display-4">{jumlah}</h1>
      <p>Target: {target} Unit</p>

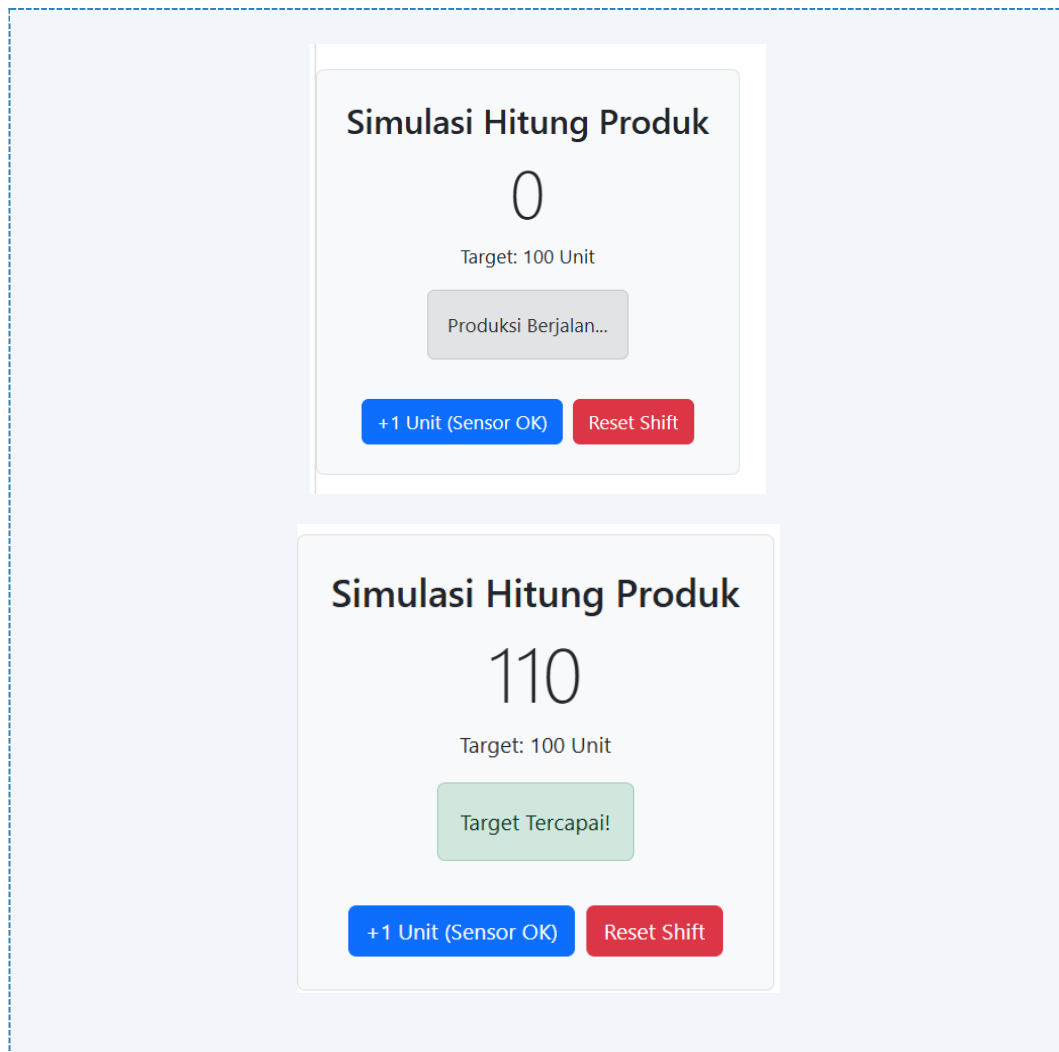
      {/* Conditional Rendering: Tampilkan pesan jika
target tercapai */}
      {jumlah >= target ? (
        <div className="alert alert-success d-inline-
block">Target Tercapai!</div>
      ) : (
        <div className="alert alert-secondary d-inline-
block">Produksi Berjalan...</div>
      )}
      <div className="mt-3">
```

```
        <button className="btn btn-primary me-2"
onClick={tambahProduksi}>
            +1 Unit (Sensor OK)
        </button>
        <button className="btn btn-danger"
onClick={reset}>
            Reset Shift
        </button>
    </div>
</div>
);
}
export default CounterProduksi;

//Di file app.jsx
<div
    className="card border-0 mb-4"
    style={{
        borderRadius: '16px',
        boxShadow: '0 8px 20px rgba(0,0,0,0.08)'
    }}
>
    <div className="card-body p-4 text-center">

        <div
            style={{
                background: '#f8fafc',
                borderRadius: '12px',
                padding: '20px'
            }}
        >
            <CounterProduksi />
        </div>
    </div>
</div>
```

Screenshot Hasil:



Gambar E.1 – Hasil Pembuatan komponen CounterProduksi.jsx

E.2 Langkah 2 – Menggunakan useEffect untuk Real-time Clock

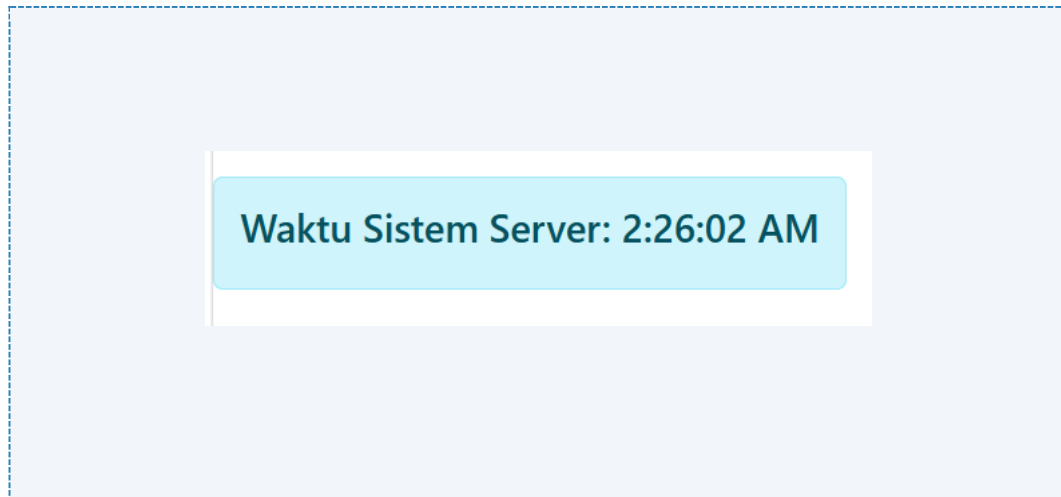
```
/* ===== KODE / OUTPUT LANGKAH 2 ===== */
import React, { useState, useEffect } from 'react';

function JamDigital() {
  const [waktu, setWaktu] = useState(new Date());
  // useEffect berjalan sekali setelah komponen dirender
  pertama kali
  useEffect(() => {
    // Membuat interval timer
    const timerID = setInterval(() => {
      setWaktu(new Date()); // Update state waktu setiap
detik
    }, 1000);
    // Cleanup function: Dijalankan saat komponen
dihapus/hancur
  });
}
```

```
// Penting untuk mencegah memory leak
return () => {
  clearInterval(timerID);
};
}, []); // Array kosong [] artinya hanya dijalankan sekali
saat mount
return (
  <div className="alert alert-info text-center">
    <h4>Waktu Sistem Server:
{waktu.toLocaleTimeString()}</h4>
  </div >
);
}
export default JamDigital;

//File app.jsx
<div
  className="card border-0"
  style={{
    borderRadius: '16px',
    boxShadow: '0 8px 20px rgba(0,0,0,0.08)'
  }}
>
  <div className="card-body p-4 text-center">
    <div
      style={{
        background: '#f8fafc',
        borderRadius: '12px',
        padding: '20px'
      }}
    >
      <JamDigital />
    </div>
  </div>
</div>
```

Screenshot Hasil:



Gambar E.2 – Hasil Pembuatan komponen JamDigital.jsx

E.3 Langkah 3 – Contoh Form Input State

```
/* ===== KODE / OUTPUT LANGKAH 3 ===== */
// 1. Import React dan useState
import React, { useState } from 'react';

// 2. Komponen Fungsi
function KartuMesin(props) {

  // Menerima data dari props
  const namaMesin = props.nama;
  const status = props.status;
  const produksi = props.produksi;

  // State lokal untuk status
  const [statusLokal, setStatusLokal] = useState(status);

  // Logika penentuan warna badge berdasarkan statusLokal
  let badgeColor = 'bg-secondary';
  if (statusLokal === 'Running') badgeColor = 'bg-success';
  if (statusLokal === 'Stop') badgeColor = 'bg-danger';
  if (statusLokal === 'Maintenance') badgeColor = 'bg-warning';

  return (
    <div className="card shadow-sm p-3 mb-3">
      <div className="card-body">
        <h5 className="card-title">{namaMesin}</h5>

        <span className={`badge ${badgeColor}`}>
          {statusLokal}
        </span>
      </div>
    </div>
  );
}
```



```

        <hr />

        <p>
            Produksi Saat Ini:
<strong>{produksi}</strong> Unit
        </p>

        <div className="mt-2">
            <select
                className="form-select form-select-sm"
                value={statusLokal}
                onChange={(e) =>
setStatusLokal(e.target.value)}
            >
                <option value="Running">Running</option>
                <option value="Stop">Stop</option>
                <option
value="Maintenance">Maintenance</option>
            </select>
        </div>
    </div>
</div>
    );
}

// 3. Export
export default KartuMesin;

// Di app.jsx
<div
    className="card border-0 mb-5"
    style={{
        borderRadius: '16px',
        boxShadow: '0 8px 20px rgba(0,0,0,0.08)'
    }}
>
    <div className="card-body p-4">
        <h3 className="text-center fw-semibold mb-4">
            Praktik 3 (Pembuatan form input state pada
Monitoring Lini Produksi A)
        </h3>

        <div className="row g-4">
            <div className="col-md-4">
                <div className="h-100 hover-card">

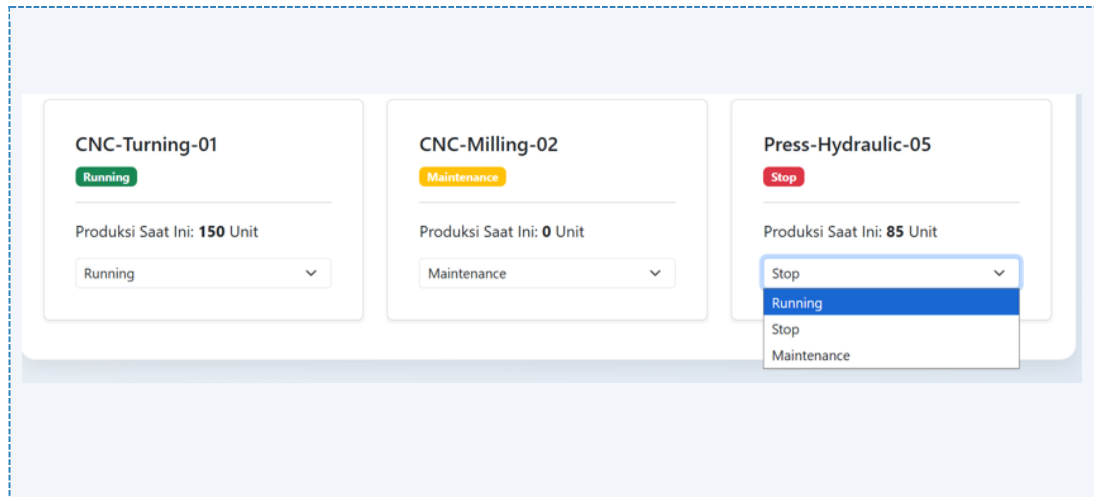
```

```
        <KartuMesin
            nama="CNC-Turning-01"
            status="Running"
            produksi={150}
        />
    </div>
</div>

<div className="col-md-4">
    <div className="h-100 hover-card">
        <KartuMesin
            nama="CNC-Milling-02"
            status="Maintenance"
            produksi={0}
        />
    </div>
</div>

<div className="col-md-4">
    <div className="h-100 hover-card">
        <KartuMesin
            nama="Press-Hydraulic-05"
            status="Stop"
            produksi={85}
        />
    </div>
</div>
</div>
</div>
```

Screenshot Hasil:



Gambar E.3– Hasil Pembuatan Form Input State

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Dependency Array useEffect

Pertanyaan / Instruksi Latihan:

Modifikasi `JamDigital`. Tambahkan input text untuk memasukkan nama kota. Gunakan `useEffect` untuk mengubah `document.title` menjadi "Jam [Nama Kota]". Gunakan state kota sebagai dependency array `[kota]`

Jawaban / Kode Solusi:

```
import React, { useState, useEffect } from 'react';

function JamDigital() {
  const [waktu, setWaktu] = useState(new Date());
  const [kota, setKota] = useState("Yogyakarta");

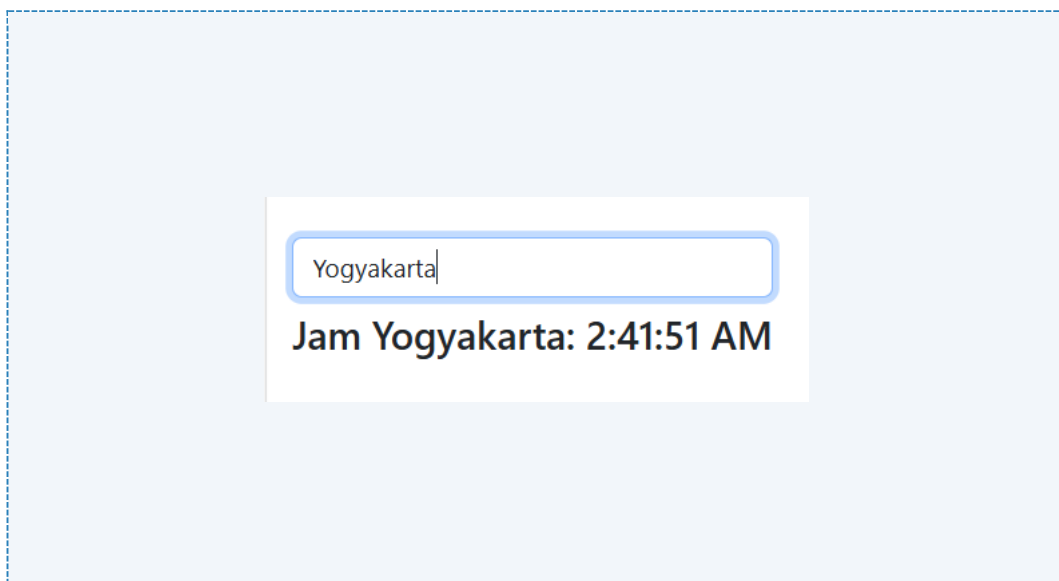
  // Effect 1: timer (mount only)
  useEffect(() => {
    const id = setInterval(() => setWaktu(new Date()), 1000);
    return () => clearInterval(id);
  }, []);

  // Effect 2: update title saat kota berubah
  useEffect(() => {
    document.title = `Jam ${kota}`;
  }, [kota]); // Dependency array berisi kota

  return (
    <div className="text-center p-3">
      <input value={kota}
        onChange={(e) => setKota(e.target.value)}
      />
    </div>
  );
}
```

```
        className="form-control mb-2" />
        <h4>Jam {kota}: {waktu.toLocaleTimeString()}</h4>
      </div>
    );
  }
  export default JamDigital;

// Di app.jsx
  <div
    className="card border-0"
    style={{
      borderRadius: '16px',
      boxShadow: '0 8px 20px rgba(0,0,0,0.08)'
    }}
  >
    <div className="card-body p-4 text-center">
      <div
        style={{
          background: '#f8fafc',
          borderRadius: '12px',
          padding: '20px'
        }}
      >
        <JamDigital />
      </div>
    </div>
  </div>
```



Gambar F.1 – Hasil Pembuatan Dependency Array useEffect

F.2 Latihan 2 – Conditional Rendering

Pertanyaan / Instruksi Latihan:

Pada `CounterProduksi`, buat tombol "Emergency Stop". Saat diklik, ubah state status menjadi "EMERGENCY", tombol "+1" menjadi disabled (`disabled`), dan tampilkan pesan merah.

Jawaban / Kode Solusi:

```
import React, { useState } from 'react';
function CounterProduksi() {
  const [jumlah, setJumlah] = useState(0);
  const [emergency, setEmergency] = useState(false);

  const handleEmergency = () => setEmergency(true);
  const reset = () => {
    setJumlah(0);
    setEmergency(false); // reset emergency juga
  };

  return (
    <div className="text-center p-4">
      <h1>{jumlah}</h1>

      {emergency && (
        <div className="alert alert-danger">
          🚨 EMERGENCY STOP AKTIF – Hubungi supervisor
        </div>
      )}

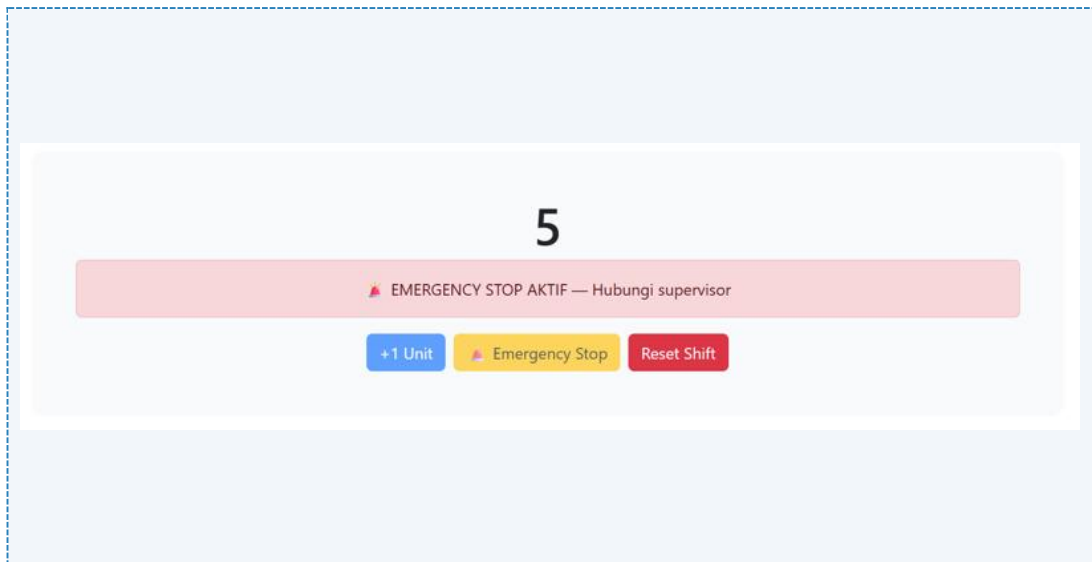
      <button className="btn btn-primary me-2"
        disabled={emergency}
        onClick={() => setJumlah(jumlah + 1)}>+1
      </button>

      <button className="btn btn-warning me-2"
        onClick={handleEmergency}
        disabled={emergency}> 🚨 Emergency Stop</button>

      <button className="btn btn-danger"
        onClick={reset}>Reset Shift</button>
    </div>
  );
}
export default CounterProduksi;
```

```
// Di app.jsx
<div
  className="card border-0 mb-4"
  style={{
    borderRadius: '16px',
    boxShadow: '0 8px 20px rgba(0,0,0,0.08)'
  }}
>
  <div className="card-body p-4 text-center">

    <div
      style={{
        background: '#f8fafc',
        borderRadius: '12px',
        padding: '20px'
      }}
    >
      <CounterProduksi />
    </div>
  </div>
</div>
```



Gambar F.2 – Hasil Pembuatan button emergency stop

F.3 Tugas Proyek Mini – Kalkulator OEE Sederhana

Deskripsi proyek mini:

- a. Buat komponen form input: `Plan Time`, `Run Time`, `Total Parts`, `Good Parts`.
- b. Gunakan State untuk menyimpan nilai-nilai ini.
- c. Hitung OEE secara otomatis (Real-time) setiap kali input berubah.
- d. Rumus $OEE = (Availability \times Performance \times Quality)$.
- e. Tampilkan hasil OEE dalam persen. Jika $OEE < 50\%$, warnanya merah. Jika $> 85\%$, warnanya hijau. Tampilkan komponen ini 3 kali di `App.jsx` dengan data karyawan berbeda (Manager, Operator, QC).

Penjelasan pendekatan/solusi yang digunakan:

Pendekatan yang digunakan dalam program ini adalah pengukuran kinerja produksi dengan metode Overall Equipment Effectiveness (OEE). Pada sistem ini, efektivitas mesin atau proses produksi dihitung berdasarkan tiga komponen utama, yaitu Availability untuk mengukur tingkat ketersediaan waktu operasi mesin dari waktu yang telah direncanakan, Performance untuk menilai kesesuaian kecepatan produksi terhadap standar, serta Quality untuk mengetahui jumlah produk yang dihasilkan dalam kondisi baik. Ketiga komponen tersebut kemudian dikombinasikan menjadi satu nilai OEE dalam bentuk persentase yang menunjukkan tingkat efektivitas proses produksi secara keseluruhan.

Dari sisi implementasi pemrograman, pendekatan yang digunakan adalah reactive programming dengan React. Setiap data seperti waktu produksi dan jumlah output disimpan dalam bentuk state, sehingga ketika terjadi perubahan nilai, sistem secara otomatis menghitung ulang nilai OEE dan memperbarui tampilan secara langsung. Hal ini membuat aplikasi lebih responsif dan memungkinkan pengguna melihat hasil secara real-time. Selain itu, digunakan pula pendekatan komponen modular, yaitu dengan memecah program menjadi beberapa bagian terpisah seperti komponen input dan komponen tampilan faktor. Pendekatan ini membuat struktur kode lebih terorganisir, mudah dipahami, serta lebih mudah dikembangkan apabila diperlukan penambahan fitur di kemudian hari. Terakhir, diterapkan aturan sederhana berupa threshold untuk mengelompokkan hasil OEE, misalnya kategori "World Class" untuk nilai tinggi dan "Need Improvement" untuk nilai rendah, sehingga hasil perhitungan dapat diinterpretasikan dengan cepat dan mudah.

Kode Utama:

```
import React, { useState } from "react";

function KalkulatorOEE() {
  // STATE
  const [planTime, setPlanTime] = useState(480);
  const [runTime, setRunTime] = useState(420);
  const [totalParts, setTotalParts] = useState(800);
  const [goodParts, setGoodParts] = useState(760);

  const standar = 2;

  // PERHITUNGAN
  const availability = planTime > 0 ? runTime / planTime : 0;
  const performance =
    runTime > 0 ? totalParts / (runTime * standar) : 0;
  const quality =
    totalParts > 0 ? goodParts / totalParts : 0;

  const oee =
    availability * performance * quality * 100;

  // WARNA DINAMIS
  let warna = "#ffc107";
  let bgStatus = "#fff3cd";
  let label = "Acceptable";

  if (oee >= 85) {
    warna = "#198754";
    bgStatus = "#d1e7dd";
    label = "World Class";
  }

  if (oee < 50) {
    warna = "#dc3545";
    bgStatus = "#f8d7da";
    label = "Need Improvement";
  }

  return (
    <div
      className="card shadow-lg border-0 p-4"
      style={{
        borderRadius: "18px",
        background:
          "linear-gradient(135deg,#f8fafc,#e9f2ff)"
      }}
    >
```



```

>
<h3 className="text-center fw-bold mb-4">
  Kalkulator OEE
</h3>

{/* INPUT */}
<div className="row g-3 mb-4">
  <InputBox
    label="Plan Time (min)"
    value={planTime}
    setValue={setPlanTime}
  />

  <InputBox
    label="Run Time (min)"
    value={runTime}
    setValue={setRunTime}
  />

  <InputBox
    label="Total Parts"
    value={totalParts}
    setValue={setTotalParts}
  />

  <InputBox
    label="Good Parts"
    value={goodParts}
    setValue={setGoodParts}
  />
</div>

<hr />

{/* OEE SCORE */}
<div
  className="text-center mb-4 p-4"
  style={{
    background: bgStatus,
    borderRadius: "14px"
  }}
>
  <h6>OEE Score</h6>

  <h1
    className="display-1 fw-bold"

```

```
        style={{
          color: warna,
          textShadow:
            "0 5px 15px rgba(0,0,0,0.15)"
        }}
      >
        {oeo.toFixed(1)}%
      </h1>

      <span
        className="badge fs-6 px-3 py-2"
        style={{
          background: warna
        }}
      >
        {label}
      </span>
    </div>

    { /* FAKTOR */ }
    <div className="row text-center">

      <FactorBox
        title="Availability"
        value={availability}
        color="#198754"
        bg="#e8f5e9"
      />

      <FactorBox
        title="Performance"
        value={performance}
        color="#0d6efd"
        bg="#e3f2fd"
      />

      <FactorBox
        title="Quality"
        value={quality}
        color="#fd7e14"
        bg="#fff3e0"
      />

    </div>
  </div>
);
```

```
}

/* COMPONENT INPUT */

function InputBox({ label, value, setValue }) {
  return (
    <div className="col-md-3">
      <label className="fw-semibold mb-1">
        {label}
      </label>

      <input
        type="number"
        className="form-control shadow-sm"
        style={{
          borderRadius: "10px"
        }}
        value={value}
        onChange={(e) =>
          setValue(
            e.target.value === ""
              ? ""
              : Number(e.target.value)
          )
        }
      />
    </div>
  );
}

/* COMPONENT FAKTOR */

function FactorBox({ title, value, color, bg }) {
  const percent = (value * 100).toFixed(1);

  return (
    <div className="col-md-4">
      <div
        className="p-3 shadow-sm"
        style={{
          borderRadius: "14px",
          background: bg,
          borderLeft:
            "6px solid " + color
        }}
      >
```

```
        <h6 className="text-muted">
          {title}
        </h6>

        <h4
          className="fw-bold"
          style={{
            color: color
          }}
        >
          {percent}%
        </h4>

        { /* PROGRESS BAR */ }

        <div className="progress mt-2">
          <div
            className="progress-bar"
            role="progressbar"
            style={{
              width: percent + "%",
              backgroundColor: color
            }}
          />
        </div>

      </div>
    </div>
  );
}

export default KalkulatorOEE;

//Di app.jsx
import React from 'react';
import KalkulatorOEE from './komponen/KalkulatorOEE';

function App() {
  return (
    <div
      style={{
        minHeight: '100vh',
        background: 'linear-gradient(135deg, #eef2f7, #dfe9f3)',
        display: 'flex',
        justifyContent: 'center'
      }}
    >
```

```
        className="py-5"
    >
    <div
        className="container"
        style={{ maxWidth: '1100px', margin: '0 auto' }}
    >

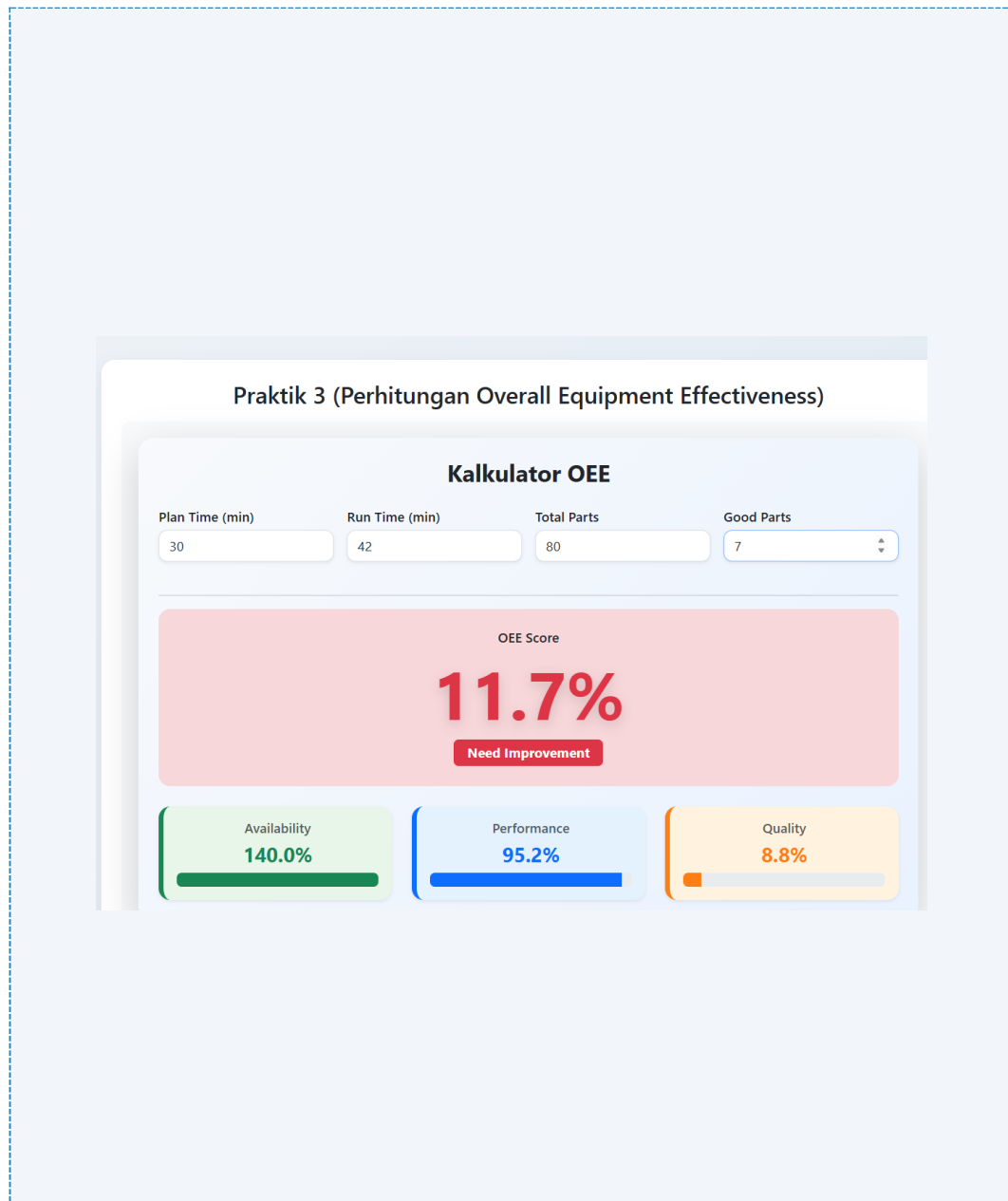
    { /* HEADER */ }
    <div className="text-center mb-5">
        <h1
            className="fw-bold"
            style={{ letterSpacing: '0.5px' }}

        <span
            className="badge px-4 py-2"
            style={{
                backgroundColor: '#2563eb',
                fontSize: '14px',
                borderRadius: '8px'
            }}
        >
            Praktik Kalkulator OEE
        </span>
    </div>

    { /* Section Kalkulator OEE */ }
    <div
        className="card border-0 mb-4"
        style={{
            borderRadius: '16px',
            boxShadow: '0 8px 20px rgba(0,0,0,0.08)'
        }}
    >
        <div className="card-body p-4">
            <div
                style={{
                    background: '#f8fafc',
                    borderRadius: '12px',
                    padding: '20px'
                }}
            >
                <KalkulatorOEE />
            </div>
        </div>
    </div>
```

```
    </div>
  </div>
);
}

export default App;
```



Gambar F.3 – Hasil Pembuatan ProyekMini Kalkulator OEE

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Secara teori, perubahan pada state seharusnya menyebabkan komponen melakukan render ulang. Apabila hasil implementasi menunjukkan bahwa tampilan langsung ikut berubah setelah data diperbarui, maka sistem tersebut telah sesuai dengan konsep dasar React. Hal ini menunjukkan bahwa mekanisme pengelolaan state telah berjalan dengan benar. Penggunaan `useEffect` juga dapat dianalisis berdasarkan waktu eksekusi prosesnya. Jika fungsi dijalankan saat komponen pertama kali ditampilkan atau ketika data mengalami perubahan, maka perilaku tersebut sudah sesuai dengan konsep lifecycle pada React. Sebaliknya, jika tidak berjalan sebagaimana mestinya, biasanya terdapat kesalahan pada dependency array atau penempatan kode. Sementara itu, conditional rendering dapat diuji dengan memberikan berbagai kondisi pada data. Jika elemen dapat muncul atau menghilang sesuai dengan aturan yang ditentukan, maka implementasinya sudah tepat. Namun, jika tampilan tidak berubah meskipun kondisi sudah berbeda, maka kemungkinan terdapat kesalahan pada logika kondisi atau pengelolaan state.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
tampilan aplikasi belum sepenuhnya menyesuaikan kondisi data yang ada, misalnya status yang ditampilkan tidak berubah meskipun nilai data telah diperbarui.	Dapat diatasi dengan mengevaluasi dan memperbaiki logika kondisi pada proses rendering

G.3 Kaitan dengan Penerapan di Industri

Hooks seperti `useEffect` umumnya dimanfaatkan untuk mengambil data dari sensor atau basis data secara berkala. Sebagai contoh, sistem dapat secara otomatis melakukan pembaruan data setiap beberapa detik guna memantau kinerja mesin. Dengan demikian, proses pengawasan menjadi lebih cepat dan akurat dibandingkan metode konvensional.

H. KESIMPULAN

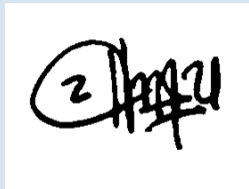
1	Hook seperti useState dan useEffect membantu mengatur logika program.
2	State digunakan untuk mengelola data yang berubah.

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	https://react.dev/learn
[3]	https://react.dev/reference/react-dom/hooks/useFormStatus
[4]	https://www.javascripttutorial.net/react-tutorial/

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Diperiksa oleh Asisten / Dosen
 Saprina Melita Sari NIM:23051430012	 Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____

